

ASP.NET Core Integration

This guide covers how to integrate ReportViewerCore with ASP.NET Core applications, including REST APIs, Razor Pages, and MVC applications.

Installation

```
dotnet add package Microsoft.Reporting.NETCore
```

Architecture Patterns

1. API Controller Pattern (Recommended)

Create a dedicated controller for report generation:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.Reporting.NETCore;
using System.Reflection;

[ApiController]
[Route("api/[controller]")]
public class ReportsController : ControllerBase
{
    private readonly ILogger<ReportsController> _logger;
    private readonly IReportService _reportService;

    public ReportsController(
        ILogger<ReportsController> logger,
        IReportService reportService)
    {
        _logger = logger;
        _reportService = reportService;
    }

    [HttpGet("invoice/{invoiceId}/pdf")]
    public IActionResult GetInvoicePdf(int invoiceId)
    {
        try
        {
            var data = _reportService.GetInvoiceData(invoiceId);
            var pdf = GenerateInvoiceReport(data, "PDF");

            return File(pdf, "application/pdf", $"Invoice_{invoiceId}.pdf");
        }
        catch (Exception ex)
        {
        }
    }
}
```

```

        _logger.LogError(ex, "Error generating invoice PDF");
        return StatusCode(500, "Error generating report");
    }
}

[HttpGet("invoice/{invoiceId}/excel")]
public IActionResult GetInvoiceExcel(int invoiceId)
{
    var data = _reportService.GetInvoiceData(invoiceId);
    var excel = GenerateInvoiceReport(data, "EXCELOPENXML");

    return File(excel,
        "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        $"Invoice_{invoiceId}.xlsx");
}

private byte[] GenerateInvoiceReport(InvoiceData data, string format)
{
    using var report = new LocalReport();

    // Load RDLC from embedded resource
    using var rdlcStream = Assembly.GetExecutingAssembly()
        .GetManifestResourceStream("YourApp.Reports.Invoice.rdlc");
    report.LoadReportDefinition(rdlcStream);

    // Add data sources
    report.DataSources.Add(new ReportDataSource("InvoiceHeader", new[] {
data.Header }));
    report.DataSources.Add(new ReportDataSource("InvoiceItems",
data.Items));

    // Set parameters
    report.SetParameters(new[] {
        new ReportParameter("InvoiceNumber", data.Header.InvoiceNumber),
        new ReportParameter("InvoiceDate",
data.Header.Date.ToString("MM/dd/yyyy"))
    });

    // Render
    return report.Render(format);
}
}

```

2. Service Layer Pattern

Create a reusable service for report generation:

```

public interface IReportGeneratorService
{

```

```

        byte[] GenerateReport<T>(string reportName, IEnumerable<T> data,
            Dictionary<string, string> parameters, string format = "PDF");
    }

    public class ReportGeneratorService : IReportGeneratorService
    {
        private readonly IWebHostEnvironment _environment;
        private readonly ILogger<ReportGeneratorService> _logger;

        public ReportGeneratorService(
            IWebHostEnvironment environment,
            ILogger<ReportGeneratorService> logger)
        {
            _environment = environment;
            _logger = logger;
        }

        public byte[] GenerateReport<T>(
            string reportName,
            IEnumerable<T> data,
            Dictionary<string, string> parameters,
            string format = "PDF")
        {
            try
            {
                using var report = new LocalReport();

                // Load report from file system
                var reportPath = Path.Combine(
                    _environment.ContentRootPath,
                    "Reports",
                    $"{reportName}.rdlc");

                using var stream = new FileStream(reportPath, FileMode.Open);
                report.LoadReportDefinition(stream);

                // Add data source
                report.DataSources.Add(new ReportDataSource("DataSet1", data));

                // Set parameters
                if (parameters != null)
                {
                    var reportParams = parameters
                        .Select(p => new ReportParameter(p.Key, p.Value))
                        .ToArray();
                    report.SetParameters(reportParams);
                }

                // Render
                return report.Render(format);
            }
            catch (Exception ex)
            {

```

```

        _logger.LogError(ex, $"Error generating report: {reportName}");
        throw;
    }
}
}

```

Register the service in `Program.cs`:

```
builder.Services.AddScoped<IReportGeneratorService, ReportGeneratorService>();
```

3. Razor Pages Pattern

Use reports in Razor Pages:

```

// Pages/Reports/Invoice.cshtml.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.Reporting.NETCore;

public class InvoiceModel : PageModel
{
    [BindProperty(SupportsGet = true)]
    public int InvoiceId { get; set; }

    [BindProperty(SupportsGet = true)]
    public string Format { get; set; } = "PDF";

    private readonly IReportGeneratorService _reportService;

    public InvoiceModel(IReportGeneratorService reportService)
    {
        _reportService = reportService;
    }

    public IActionResult OnGet()
    {
        // Get data from your data service
        var invoiceData = GetInvoiceData(InvoiceId);

        var parameters = new Dictionary<string, string>
        {
            ["InvoiceId"] = InvoiceId.ToString(),
            ["GeneratedDate"] = DateTime.Now.ToString("yyyy-MM-dd")
        };

        var reportBytes = _reportService.GenerateReport(
            "Invoice",
            invoiceData,

```

```

        parameters,
        Format.ToUpper());

    return Format.ToUpper() switch
    {
        "PDF" => File(reportBytes, "application/pdf",
            $"Invoice_{InvoiceId}.pdf"),
        "EXCEL" => File(reportBytes,
            "application/vnd.openxmlformats-
            officedocument.spreadsheetml.sheet",
            $"Invoice_{InvoiceId}.xlsx"),
        "WORD" => File(reportBytes,
            "application/vnd.openxmlformats-
            officedocument.wordprocessingml.document",
            $"Invoice_{InvoiceId}.docx"),
        _ => File(reportBytes, "application/pdf",
            $"Invoice_{InvoiceId}.pdf")
    };
}

```

Inline HTML Rendering

For displaying reports directly in the browser:

```

[HttpGet("invoice/{invoiceId}/preview")]
public IActionResult PreviewInvoice(int invoiceId)
{
    var data = _reportService.GetInvoiceData(invoiceId);

    using var report = new LocalReport();
    LoadReport(report, data);

    // Render as HTML5
    var html = report.Render("HTML5");

    return Content(Encoding.UTF8.GetString(html), "text/html");
}

```

HTML5 with Embedded Images

The HTML5 renderer embeds images as data URLs, making it self-contained:

```

// No additional configuration needed - images are automatically embedded
byte[] html = report.Render("HTML5");

```

Handling Large Reports

For large reports, use streaming:

```
[HttpGet("report/large/{id}")]
public IActionResult GetLargeReport(int id)
{
    var data = GetLargeDataSet(id);

    using var report = new LocalReport();
    LoadReport(report, data);

    // Use memory-efficient rendering
    var stream = new MemoryStream();

    report.Render("PDF", null, (name, extension, encoding, mimeType, willSeek)
=>
    {
        return stream;
    }, out Warning[] warnings);

    stream.Position = 0;

    return File(stream.ToArray(), "application/pdf", "large-report.pdf");
}
```

Dependency Injection Configuration

Minimal API Setup (.NET 6+)

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

// Add services
builder.Services.AddControllers();
builder.Services.AddScoped<IReportGeneratorService, ReportGeneratorService>();

var app = builder.Build();

app.MapControllers();
app.Run();
```

Traditional Startup

```
// Startup.cs
public class Startup
{
    public void ConfigureServices(IServiceCollection services)
    {

```

```

        services.AddControllers();
        services.AddScoped<IReportGeneratorService, ReportGeneratorService>();
    }

    public void Configure(IApplicationBuilder app)
    {
        app.UseRouting();
        app.UseEndpoints(endpoints =>
        {
            endpoints.MapControllers();
        });
    }
}

```

Report Caching

Improve performance by caching compiled reports:

```

public class CachedReportService : IReportGeneratorService
{
    private readonly IMemoryCache _cache;
    private readonly IWebHostEnvironment _environment;

    public CachedReportService(
        IMemoryCache cache,
        IWebHostEnvironment environment)
    {
        _cache = cache;
        _environment = environment;
    }

    public byte[] GenerateReport<T>(
        string reportName,
        IEnumerable<T> data,
        Dictionary<string, string> parameters,
        string format = "PDF")
    {
        var cacheKey = $"report-definition-{reportName}";

        // Try to get cached report definition
        if (!_cache.TryGetValue(cacheKey, out byte[] reportDefinition))
        {
            // Load and cache the report definition
            var reportPath = Path.Combine(
                _environment.ContentRootPath,
                "Reports",
                $"{reportName}.rdlc");

            reportDefinition = File.ReadAllBytes(reportPath);
        }
    }
}

```

```

        _cache.Set(cacheKey, reportDefinition, TimeSpan.FromHours(1));
    }

    using var report = new LocalReport();
    using var stream = new MemoryStream(reportDefinition);
    report.LoadReportDefinition(stream);

    // Add data and render
    report.DataSources.Add(new ReportDataSource("DataSet1", data));

    if (parameters != null)
    {
        var reportParams = parameters
            .Select(p => new ReportParameter(p.Key, p.Value))
            .ToArray();
        report.SetParameters(reportParams);
    }

    return report.Render(format);
}

```

Error Handling

Implement comprehensive error handling:

```

[HttpGet("report/{reportName}")]
public IActionResult GenerateReport(string reportName, [FromQuery] string
format = "PDF")
{
    try
    {
        var data = GetReportData(reportName);
        var reportBytes = _reportService.GenerateReport(reportName, data, null,
format);

        var (mimeType, extension) = format.ToUpper() switch
        {
            "PDF" => ("application/pdf", "pdf"),
            "EXCELOPENXML" => ("application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet", "xlsx"),
            "WORDOPENXML" => ("application/vnd.openxmlformats-
officedocument.wordprocessingml.document", "docx"),
            _ => ("application/pdf", "pdf")
        };

        return File(reportBytes, mimeType, $"{reportName}.{extension}");
    }
    catch (FileNotFoundException ex)
    {

```



```

        _logger.LogError(ex, $"Report not found: {reportName}");
        return NotFound($"Report '{reportName}' not found");
    }
    catch (LocalProcessingException ex)
    {
        _logger.LogError(ex, "Report processing error");
        return StatusCode(500, "Error processing report");
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Unexpected error generating report");
        return StatusCode(500, "An error occurred while generating the
report");
    }
}

```

Authentication & Authorization

Secure your report endpoints:

```

[Authorize]
[ApiController]
[Route("api/[controller]")]
public class ReportsController : ControllerBase
{
    [HttpGet("financial/{reportId}")]
    [Authorize(Roles = "Finance,Admin")]
    public IActionResult GetFinancialReport(int reportId)
    {
        // Only Finance and Admin roles can access
        var report = GenerateFinancialReport(reportId);
        return File(report, "application/pdf");
    }

    [HttpGet("employee/{employeeId}")]
    public IActionResult GetEmployeeReport(int employeeId)
    {
        // Verify user can access this employee's data
        var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
        if (!CanAccessEmployee(userId, employeeId))
        {
            return Forbid();
        }

        var report = GenerateEmployeeReport(employeeId);
        return File(report, "application/pdf");
    }
}

```

CORS Configuration

Enable CORS for cross-origin requests:

```
// Program.cs
builder.Services.AddCors(options =>
{
    options.AddPolicy("ReportPolicy", policy =>
    {
        policy.WithOrigins("https://your-frontend-domain.com")
            .AllowAnyMethod()
            .AllowAnyHeader();
    });
});

var app = builder.Build();
app.UseCors("ReportPolicy");
```

Example: Complete API Implementation

```
// Complete working example
[ApiController]
[Route("api/[controller]")]
public class ReportsController : ControllerBase
{
    private readonly IReportGeneratorService _reportService;
    private readonly IReportDataService _dataService;

    public ReportsController(
        IReportGeneratorService reportService,
        IReportDataService dataService)
    {
        _reportService = reportService;
        _dataService = dataService;
    }

    [HttpPost("sales")]
    public IActionResult GenerateSalesReport(
        [FromBody] SalesReportRequest request)
    {
        var data = _dataService.GetSalesData(
            request.StartDate,
            request.EndDate,
            request.Region);

        var parameters = new Dictionary<string, string>
        {
            ["StartDate"] = request.StartDate.ToString("yyyy-MM-dd"),
            ["EndDate"] = request.EndDate.ToString("yyyy-MM-dd"),
        }
    }
}
```

```

        ["Region"] = request.Region,
        ["GeneratedBy"] = User.Identity.Name
    };

    var report = _reportService.GenerateReport(
        "SalesReport",
        data,
        parameters,
        request.Format ?? "PDF");

    var contentType = GetContentType(request.Format);
    var fileName = $"SalesReport_{DateTime.Now:yyyyMMdd}.
{GetExtension(request.Format)}";

    return File(report, contentType, fileName);
}

private string GetContentType(string format) => format?.ToUpper() switch
{
    "EXCEL" or "EXCELOPENXML" => "application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet",
    "WORD" or "WORDOPENXML" => "application/vnd.openxmlformats-
officedocument.wordprocessingml.document",
    _ => "application/pdf"
};

private string GetExtension(string format) => format?.ToUpper() switch
{
    "EXCEL" or "EXCELOPENXML" => "xlsx",
    "WORD" or "WORDOPENXML" => "docx",
    _ => "pdf"
};

}

public class SalesReportRequest
{
    public DateTime StartDate { get; set; }
    public DateTime EndDate { get; set; }
    public string Region { get; set; }
    public string Format { get; set; } = "PDF";
}

```

Next Steps

- [Blazor Integration](#) - Use reports in Blazor applications
- [API vs Standalone Mode](#) - Understand different deployment patterns
- [Advanced Topics](#) - Subreports, custom formatting, and more